

Labsheet 5

1. .env

```
PORT=4000
```

2. package.json

```
{
  "name": "labsheet5",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.19.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

3. public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Labsheet 5 - Express Lab</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Labsheet 5 - Express Lab Running</h1>
  <p>Welcome to the Express.js Server</p>
</body>
</html>
```

4. public/style.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  text-align: center;
  padding-top: 50px;
  color: #333;
}

h1 {
  color: #2c3e50;
}
```

5. routes/students.js

```
const express = require('express');
const router = express.Router();

// Part C & F: GET /api/students/:id
router.get('/:id', (req, res) => {
  const { id } = req.params;
  res.json({ id, message: "Student details" });
});

module.exports = router;
```

6. routes/books.js

```
const express = require('express');
const router = express.Router();

// In-memory data store for Books
let books = [
  { id: 1, title: "Node.js Guide", author: "John Doe" },
  { id: 2, title: "Express Deep Dive", author: "Jane Smith" }
];

// GET /api/books - Get all books
router.get('/', (req, res) => {
  res.json(books);
});

// GET /api/books/:id - Get single book
router.get('/:id', (req, res, next) => {
  const id = parseInt(req.params.id);
```

```

    const book = books.find(b => b.id === id);
    if (!book) {
      const error = new Error("Book not found");
      error.status = 404;
      return next(error);
    }
    res.json(book);
  });

// POST /api/books - Add a new book
router.post('/', (req, res) => {
  const { title, author } = req.body;
  const newBook = { id: books.length + 1, title, author };
  books.push(newBook);
  res.status(201).json(newBook);
});

// PUT /api/books/:id - Update a book
router.put('/:id', (req, res, next) => {
  const id = parseInt(req.params.id);
  const bookIndex = books.findIndex(b => b.id === id);
  if (bookIndex === -1) {
    const error = new Error("Book not found for update");
    error.status = 404;
    return next(error);
  }
  books[bookIndex] = { id, ...req.body };
  res.json(books[bookIndex]);
});

// DELETE /api/books/:id - Delete a book
router.delete('/:id', (req, res, next) => {
  const id = parseInt(req.params.id);
  const bookIndex = books.findIndex(b => b.id === id);
  if (bookIndex === -1) {
    const error = new Error("Book not found for deletion");
    error.status = 404;
    return next(error);
  }
  const deletedBook = books.splice(bookIndex, 1);
  res.json({ message: "Book deleted successfully", book: deletedBook[0] });
});

module.exports = router;

```

7. routes/members.js

```

const express = require('express');
const router = express.Router();

// In-memory data store for Members
let members = [
  { id: 1, name: "Rahul Sharma", role: "Student" },
  { id: 2, name: "Priya Singh", role: "Faculty" }
];

// GET /api/members - Get all members
router.get('/', (req, res) => {
  res.json(members);
});

// GET /api/members/:id - Get single member
router.get('/:id', (req, res, next) => {
  const id = parseInt(req.params.id);
  const member = members.find(m => m.id === id);
  if (!member) {
    const error = new Error("Member not found");
    error.status = 404;
    return next(error);
  }
  res.json(member);
});

// POST /api/members - Add a new member
router.post('/', (req, res) => {
  const { name, role } = req.body;
  const newMember = { id: members.length + 1, name, role };
  members.push(newMember);
  res.status(201).json(newMember);
});

// PUT /api/members/:id - Update a member
router.put('/:id', (req, res, next) => {
  const id = parseInt(req.params.id);
  const memberIndex = members.findIndex(m => m.id === id);
  if (memberIndex === -1) {
    const error = new Error("Member not found for update");

```

```

        error.status = 404;
        return next(error);
    }
    members[memberIndex] = { id, ...req.body };
    res.json(members[memberIndex]);
});

// DELETE /api/members/:id - Delete a member
router.delete('/:id', (req, res, next) => {
    const id = parseInt(req.params.id);
    const memberIndex = members.findIndex(m => m.id === id);
    if (memberIndex === -1) {
        const error = new Error("Member not found for deletion");
        error.status = 404;
        return next(error);
    }
    const deletedMember = members.splice(memberIndex, 1);
    res.json({ message: "Member deleted successfully", member: deletedMember[0] });
});

module.exports = router;

```

8. index.js

```

require('dotenv').config();
const express = require('express');
const path = require('path');

const studentRoutes = require('./routes/students');
const bookRoutes = require('./routes/books');
const memberRoutes = require('./routes/members');

const app = express();
const PORT = process.env.PORT || 4000;

// Middleware for parsing JSON and form data
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Serve static files from public folder
app.use(express.static(path.join(__dirname, 'public')));

// Global Custom Logger Middleware
app.use((req, res, next) => {
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
    next();
});

// Admin API Key Auth Middleware
const checkApiKey = (req, res, next) => {
    const apiKey = req.headers['x-api-key'];
    if (!apiKey) {
        return res.status(403).json({ error: "403 Forbidden: Missing x-api-key header" });
    }
    next();
};

// Part A & B: Basic Routes
app.get('/', (req, res) => {
    res.send("Express Lab Running");
});

app.get('/about', (req, res) => {
    res.json({ name: "Ashish Singh Rawat", rollNumber: "2024001" });
});

app.get('/courses', (req, res) => {
    res.json(["Full Stack Web Development", "Data Structures", "Database Management"]);
});

app.post('/echo', (req, res) => {
    res.json(req.body);
});

// Part C: Route Parameters & Query Strings
app.get('/search', (req, res) => {
    const { name, age } = req.query;
    res.json({ name, age });
});

app.get('/products/:category/:id', (req, res) => {
    const { category, id } = req.params;
    res.json({ category, id });
});

// Part D: Protected Route
app.get('/admin/dashboard', checkApiKey, (req, res) => {

```

```

    res.json({ message: "Welcome to Admin Dashboard" });
  });

// Part E: Forms & JSON Submissions
app.post('/register', (req, res) => {
  const { name, email, password } = req.body;
  res.json({ message: `Registration successful for ${name}` });
});

app.post('/contact', (req, res) => {
  console.log("Contact Form Received:", req.body);
  res.send("Thank you for contacting us!");
});

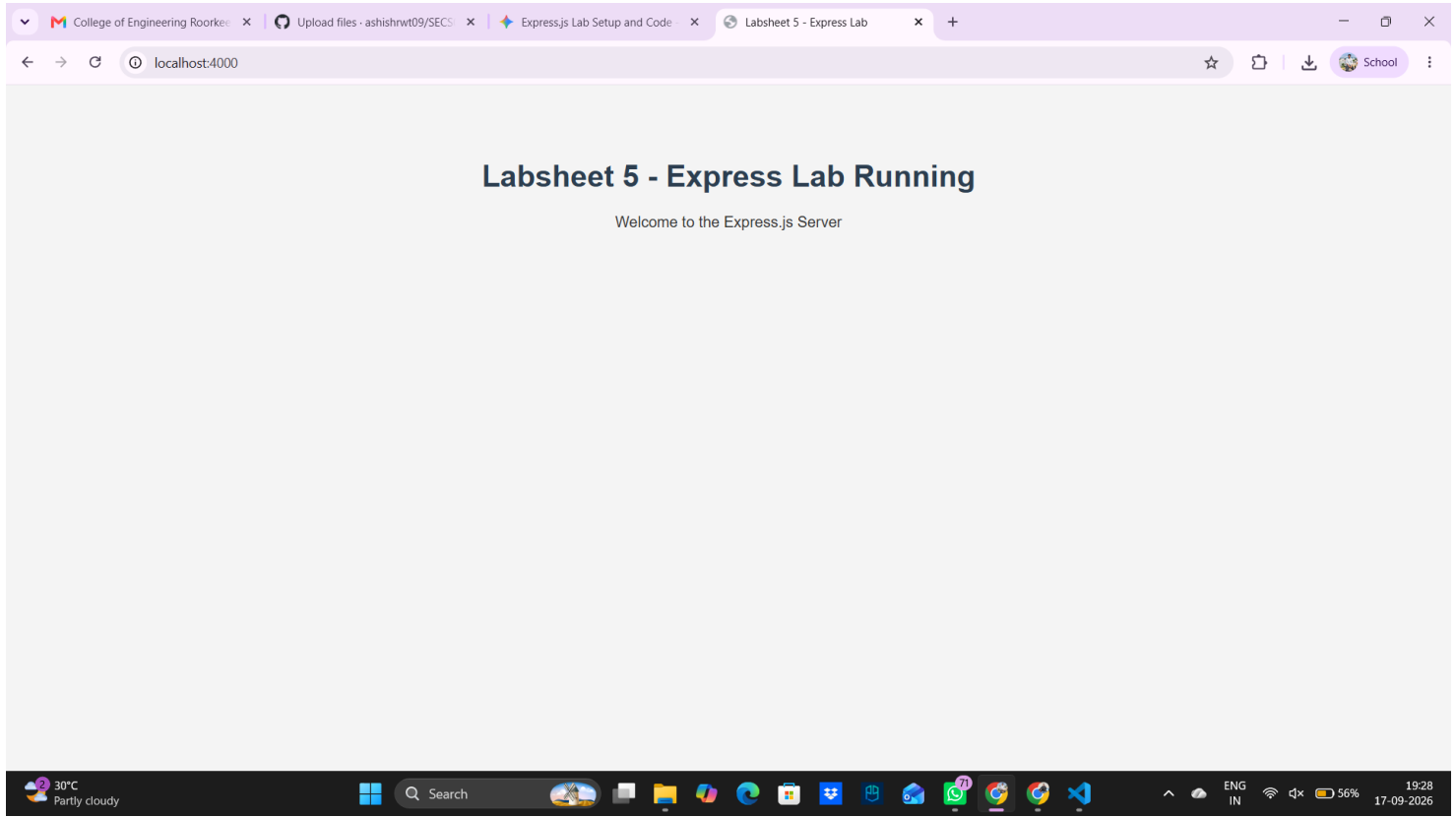
// Part F & J: Router Mounting
app.use('/api/students', studentRoutes);
app.use('/api/books', bookRoutes);
app.use('/api/members', memberRoutes);

// Part H: 404 Route Handler
app.use((req, res, next) => {
  res.status(404).json({ error: "Route not found" });
});

// Part H: Centralized Error Handler
app.use((err, req, res, next) => {
  console.error("Error Log:", err.message);
  const statusCode = err.status || 500;
  res.status(statusCode).json({
    error: {
      message: err.message || "Internal Server Error",
      status: statusCode
    }
  });
});

app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});

```



```
{ "name": "Ashish Singh Rawat", "rollNumber": "2024001" }
```

{ "error": "Route not found" }

Thunder Client

New Request

ActivityCollectionsEnv

filter activity

GETlocalhost:4000/api/booksjust now

GETlocalhost:4000/api/booksjust now

POSTlocalhost:5000/api/resume/upload6 months ago

POSTlocalhost:5000/api/auth/login6 months ago

POSTlocalhost:5000/api/auth/register6 months ago

POSTlocalhost:5000/api/auth/login6 months ago

POSTlocalhost:5000/api/auth/register6 months ago

POSTlocalhost:5000/api/auth/register6 months ago

GETlocalhost:5000/api/todos7 months ago

GETlocalhost:5000/api/todos7 months ago

GETlocalhost:3000/notes7 months ago

GETlocalhost:4000/api/books999

Send

QueryHeaders4AuthBody1TestsPre Run

checkboxchecked

User-AgentThunder Client (https://www.thunderclient.com)

checkboxchecked

headerx-api-key

checkboxchecked

key12345

checkboxunchecked

headervalue

Status: 404 Not FoundSize: 51 BytesTime: 19 msResponse

1{

2 "error": {

3 "message": "Book not found",

4 "status": 404

5 }

6 }

ResponseChart

Problems3OutputDebug ConsoleTerminalPorts

create mode 100644 package.json

create mode 100644 public/index.html

create mode 100644 public/style.css

create mode 100644 routes/books.js

create mode 100644 routes/members.js

create mode 100644 route...

remote: Permission to ash... denied to Ashish-singh-rawat-09.

fatal: unable to access 'https://github.com/ashishrwt09/fullstack.git/': The requested URL returned error: 403

PS C:\Users\rawat\Desktop\Full Stack Clg lab\labsheet>

powershell

powershell

powershell

Code labsh...

ChatSessions

Update

Tip: Use ./create-agent to scaffold a custom agent for your workflow.

+ index.js

Describe what to build

+ AgentA...Balance⇌↓↑

LocalDefault permissions

main*

30°CPartly cloudy

Search

ENGIN

52%

19:3617-09-2026

Thunder Client

New Request

ActivityCollectionsEnv

filter activity

GETlocalhost:4000/api/books

just now

GETlocalhost:4000/api/books

just now

POSTlocalhost:5000/api/resume/upload

6 months ago

POSTlocalhost:5000/api/auth/login

6 months ago

POSTlocalhost:5000/api/auth/register

6 months ago

POSTlocalhost:5000/api/auth/login

6 months ago

POSTlocalhost:5000/api/auth/register

6 months ago

POSTlocalhost:5000/api/auth/register

6 months ago

GETlocalhost:5000/api/todos

7 months ago

GETlocalhost:5000/api/todos

7 months ago

GETlocalhost:3000/notes

7 months ago

GETlocalhost:4000/api/books/999

Send

QueryHeadersAuthBodyTestsPre Run

User-AgentThunder Client (https://www.thunderclient.com)

headerx-api-key

key12345

headervalue

Status: 404 Not FoundSize: 51 BytesTime: 19 ms

Response

1{

2"error": {

3"message": "Book not found",

4"status": 404

5}

6}

ResponseChart

ProblemsOutputDebug ConsoleTerminalPorts

PS C:\Users\rawat\Desktop> # 1. Main project folder ke andar jao

>> cd "Full Stack Clg lab"

>>

>> # 2. labsheet5 ke andar se cached Git entry ko hatao ...

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Everything up-to-date

PS C:\Users\rawat\Desktop\Full Stack Clg lab>

powershell

powershell

Code

ChatSessions

Update

Tip: Use /create-agent to scaffold a custom agent for your workflow.

+index.js

Describe what to build

+AgentA...Balance=↑51%

LocalDefault permissions

main

62

30°C

Partly cloudy

Search

ENG

IN

51%

19:39

17-09-2026

The image shows a Windows desktop environment. The primary application window is Thunder Client, which is configured with the URL 'http://localhost:4000/api/books/999'. The 'Headers' tab is active, showing 'User-Agent' as 'Thunder Client (https://www.thunderclient.com)', 'header' as 'x-api-key', and 'key' as '12345'. The 'Status' bar indicates a '404 Not Found' response with a size of 51 Bytes and a time of 19 ms. The response body is a JSON object:

```
{  "error": {    "message": "Book not found",    "status": 404  }}
```

. Below the response, the 'Terminal' tab is open, showing a command prompt session where a git commit command failed with the error: 'The token '&&' is not a valid statement separator in this version.' The Windows taskbar at the bottom displays the system clock as 19:41 on 17-09-2026, along with various system icons and open applications.